

Sprawozdanie z projektu nr 2 -
- Biometria tęczówki

Biometria

Sebastian Pergała, nr indeksu: 327301

23 kwietnia 2025

Spis treści

1	Wstęp	3
1.1	Cel projektu	3
1.2	Metodologia	3
1.3	Wykorzystane dane	4
2	Wstępna obróbka obrazów	6
3	Wykrywanie źrenicy	7
3.1	Binaryzacja	7
3.1.1	Adaptacyjny próg binaryzacji	7
3.2	Operacje morfologiczne	8
3.2.1	Zamknięcie	8
3.2.2	Otwarcie	9
3.2.3	Oczyszczenie obrazu	9
3.3	Projekcje	10
3.4	Przeniesienie na inny obraz i wyniki	11
4	Wykrywanie tęczówki	12
4.1	Nieudana próba	12
4.2	Binaryzacja	13
4.2.1	Oczyszczenie obrazu	13
4.2.2	Adaptacyjny próg binaryzacji	13
4.3	Operacje morfologiczne	13
4.3.1	Zamknięcie	13
4.3.2	Otwarcie	14
4.4	Badanie pochodnej jasności	14
4.5	Przeniesienie na inny obraz i wyniki	17
5	Kod tęczówki	18
5.1	Metodologia	18
5.2	Radialne pasy	19
5.3	Rozwinięcie pasów	20
5.4	Kompresja pasów do jednego wymiaru	21
5.5	Transformacja falkami Gabora	21
5.6	Otrzymanie kodu tęczówki	22
6	Wynik	23
6.1	Przedstawienie wyniku	23
6.2	Badanie odległości Hamminga	23
7	Aplikacja	25
8	Podsumowanie	27
9	Literatura	28

1 Wstęp

1.1 Cel projektu

Celem projektu było opracowanie metody lokalizacji tęczówki, poddanie jej segmentacji i rozwinięciu do prostokąta, a następnie otrzymanie kodu tęczówki algorytmem Daugmana.

1.2 Metodologia

Projekt wykonałem w języku Python. Poza bazowymi modułami obecnymi domyślnie w Pythonie wykorzystałem następujące:

- numpy - operacje macierzowe i inne operacje matematyczne,
- matplotlib - rysowanie wykresów,
- opencv-python - przetwarzanie obrazów,
- Pillow - zapisywanie i wyświetlanie obrazów.

W poprzednim projekcie stworzyłem aplikację służącą do przetwarzania obrazów. Algorytmy implementowane w tej aplikacji były przeze mnie od podstaw. W obecnym projekcie natomiast celem było opracowanie algorytmu wykorzystywanego w biometrii tęczówki, więc pozwoliłem sobie użyć w niektórych miejscach już wbudowanych w moduł od OpenCV funkcji do bardziej skomplikowanych operacji na obrazach.

Mój proces badawczy był podzielony na etapy, w których zajmowałem się konkretnym zagadnieniem.

0. przygotowania,
1. znajdowanie źrenicy,
2. znajdowanie tęczówki,
3. pozyskiwanie kodu tęczówki,
4. badanie odległości Hamminga,
5. stworzenie ostatecznego pipeline.

Po zakończeniu procesu badawczego skondensowałem kod do przyjaznej użytkownikowi formy w postaci aplikacji.

Szczegółowe kroki podjęte w każdym z etapów, jak i opis aplikacji, są opisane w odpowiednich sekcjach w tym dokumencie.

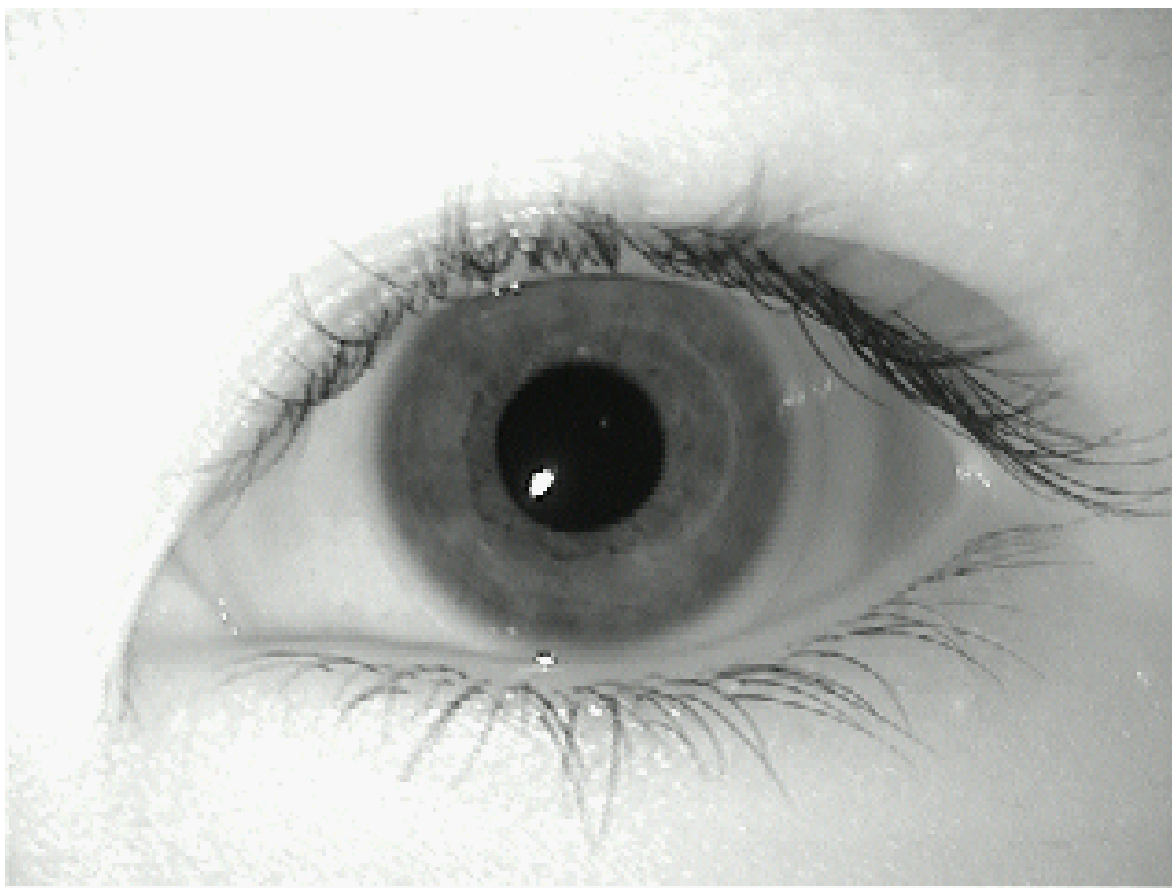
Na każdym etapie, jak i w końcowej aplikacji, program mógł operować na dowolnej liczbie zdjęć. Dodatkowo moja implementacja algorytmu do lokalizacji tęczówki i pozyskiwania jej kodu zawiera parametry, które użytkownik może w prosty sposób zmieniać zarówno w kodzie notatników, jak i w samej aplikacji, aby dostosować program do własnych potrzeb. Parametry określające rozmiary określone względem wymiaru obrazów, aby algorytm działał poprawnie bez względu na rozdzielczość zdjęcia. Założyłem, że przekazywane zdjęcia będą zawierały tylko oczy oraz ewentualnie inne elementy jak powieki. Na podstawie zbiorów danych, z którymi

miałem do czynienia w tym projekcie zauważyłem, że w osi x na ogół ta sama przestrzeń jest zajęta przez oko, natomiast w osi y czasem całą przestrzeń zajmuje oko, czasem dołączone są powieki, a w niektórych przypadkach nawet jest obecna brew. Dlatego zmienne mające do czynienia z wielkością uzależniłem od szerokości obrazu.

Na etapie pozyskiwania kodu z już wyznaczonej tęczówki postępowalem zgodnie z metodą opisaną w książce Krzysztofa Ślota; *Wybrane zagadnienia biometrii*.

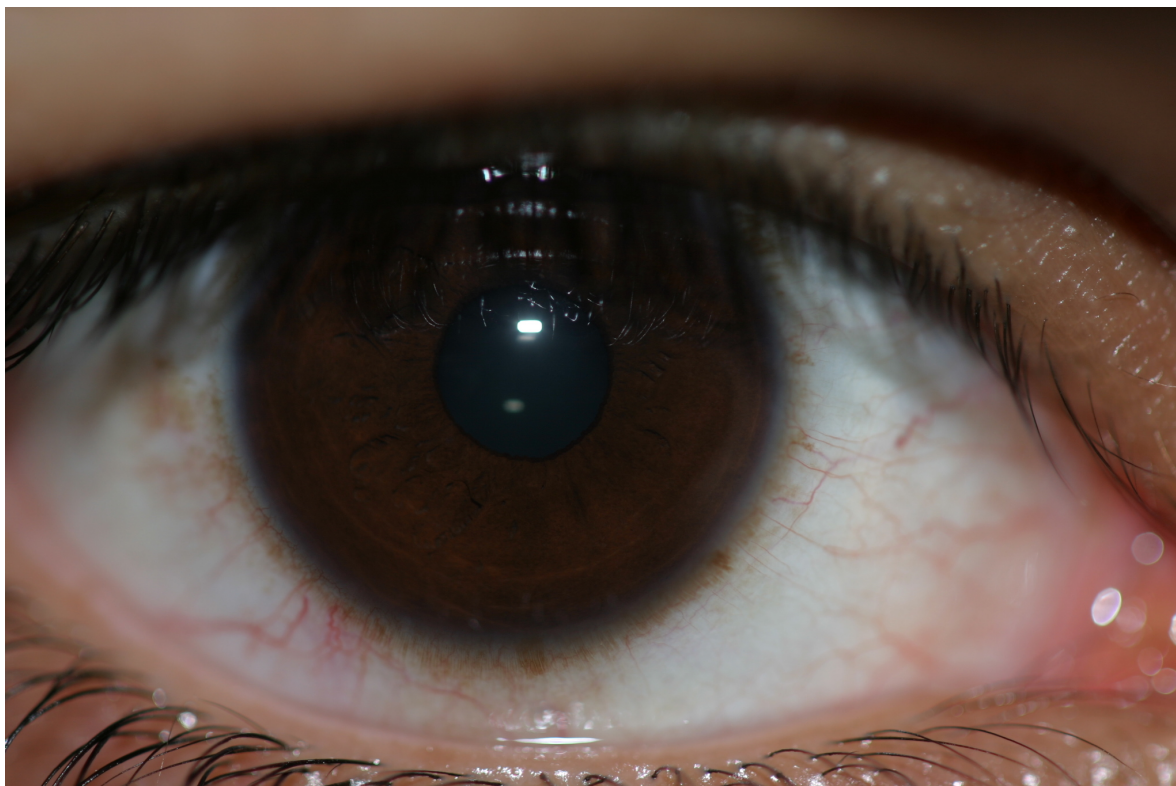
1.3 Wykorzystane dane

Do projektu wykorzystałem zbiór danych *MMU iris dataset* zawierający 5 zdjęć 320 na 240 pikseli w formacie .bmp oka lewego i 5 zdjęć oka prawego 46 różnych osób - w sumie 460 zdjęć. Obraz te, choć nie były w odcieniach szarości, nie miały informacji o kolorze. Zdjęcia były dobrej jakości, choć z uwagi na ich niewielki rozmiar, wskutek kompresji, pozyskanie dobrej jakości kodów tęczówki było niemożliwe. Niemniej parametry algorytmu doбираłem w ten sposób, aby właśnie na tym zbiorze danych otrzymać najlepsze wyniki - założeniem samego projektu była ocena działania algorytmu Daugmana na tym zbiorze danych.



Rysunek 1: Przykładowe zdjęcie ze zbioru *MMU iris dataset*

W ramach projektu otrzymałem dodatkowy zbiór danych z tęczówkami, tym razem grafiki były większych rozmiarów, bo 2048 na 1360 pikseli i były kolorowe. Niestety, sama jakość tych zdjęć była o wiele gorsza, niż w przypadku *MMU iris dataset*. W części przypadków mój algorytm działał poprawnie na dodatkowym zbiorze zdjęć, ale w ogólności wyniki były o wiele gorsze, niż dla *MMU iris dataset*. Poniżej zamieściłem jeden z najbardziej ekstremalnych przykładów zdjęcia z tego zbioru danych jeśli chodzi o brak klarowności co do granicy tęczówki.



Rysunek 2: Przykładowe zdjęcie złej jakości z innego zbioru danych

2 Wstępna obróbka obrazów

Po wczytaniu zdjęć zmieniałem ich rozmiar tak, aby na szerokość miały co najwyżej 256 pikseli, jeśli początkowo miały więcej. Ten zabieg miał na celu zwiększenie szybkości działania algorytmu. Aby nie stracić informacji o strukturze tęczówki, wyliczone granice źrenicy i tęczówki były w późniejszej fazie algorytmu nanoszone na obrazy o początkowej wielkości.

Następnie obrazy były przekształcane do odcieni szarości.



Rysunek 3: Obrazy przekształcone do odcieni szarości

Kolejnym etapem była eliminacja niedoskonałości z jednoczesnym zatrzymaniem informacji o położeniu i granicach tęczówki i źrenicy. Przeszkodę stanowiły odbłaski na powierzchni oka oraz rzęsy, zasłaniające część tęczówki. Na zdjęciach zastosowałem operację morfologiczną wyciągającą małe, jasne elementy z tła, i następnie odejmującą te elementy od oryginalnego zdjęcia. Następnie użyłem dolnoprzepustowy filtr gaussowski, aby wygładzić widoczność rzęs i innych małych elementów mogących zakłócić proces szukania granic elementów oka. Na koniec zapełniałem większe obszary z prześwietleniami stosując funkcję *inpaint* z OpenCV na rozszerzonej dylacją masce dla pikseli powyżej pewnego progu jasności.



Rysunek 4: Obrazy po usunięciu odbłasków

3 Wykrywanie źrenicy

3.1 Binarizacja

3.1.1 Adaptacyjny próg binaryzacji

Do binaryzacji dla źrenicy wykorzystałem następujący algorytm:

$$\sum_{i=0}^{h-1} \sum_{j=0}^{w-1} \frac{A(i, j)}{hw},$$

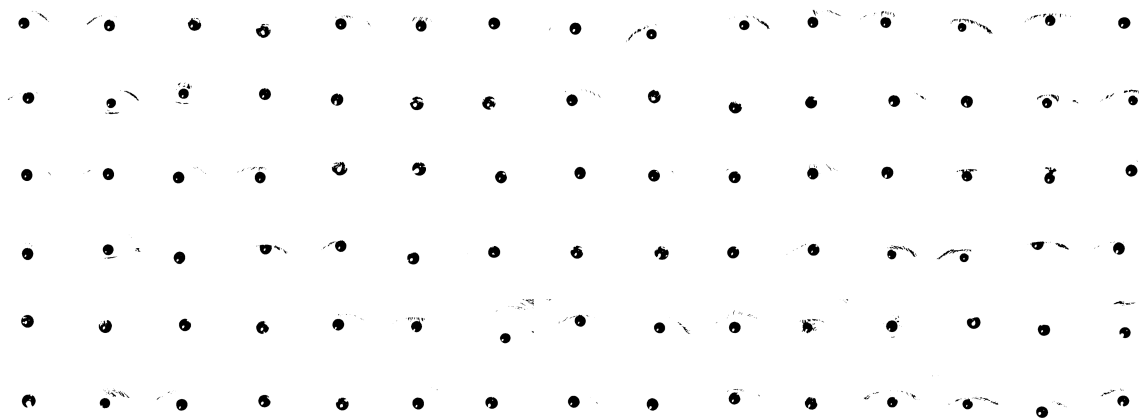
gdzie

- P - próg binaryzacji,
- h - wysokość obrazu,
- w - szerokość obrazu,
- A(i, j) - jasność piksela szarego.

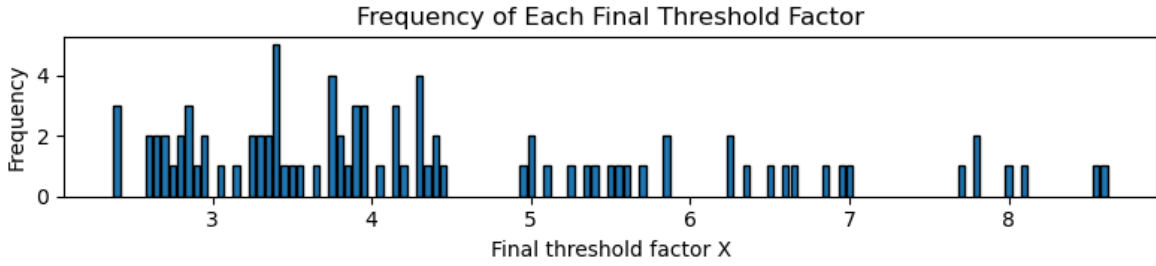
Ostateczny próg binaryzacji dla tęczówki P_I wyznaczałem jako $P_I = \frac{P}{X_I}$.

Na tym etapie celem było wyznaczenie odpowiedniej wartości parametru X_I .

Zauważyłem, że dla różnych zdjęć, różne wartości X_I dawały lepsze efekty. Jednak chciałem, aby program działał jak najlepiej niezależnie od tego, jakie dostanie zdjęcie. W tym celu zaimplementowałem adaptacyjną metodę wybierania X_I ; zmniejszałem wartość X_I o ustalony krok dla każdego zdjęcia aż do momentu, gdy frakcja pikseli białych osiągnęła odpowiedni poziom. Eksperymentalnie optymalną wartość maksymalnej dopuszczalnej frakcji pikseli białych wyznaczyłem na 0.975.



Rysunek 5: Binarizacja adaptacyjną wartością progu dla maksymalnej frakcji białych pikseli równej 0.975



Rysunek 6: Histogram wyznaczonych wartości X_I za pomocą metody adaptacyjnej

3.2 Operacje morfologiczne

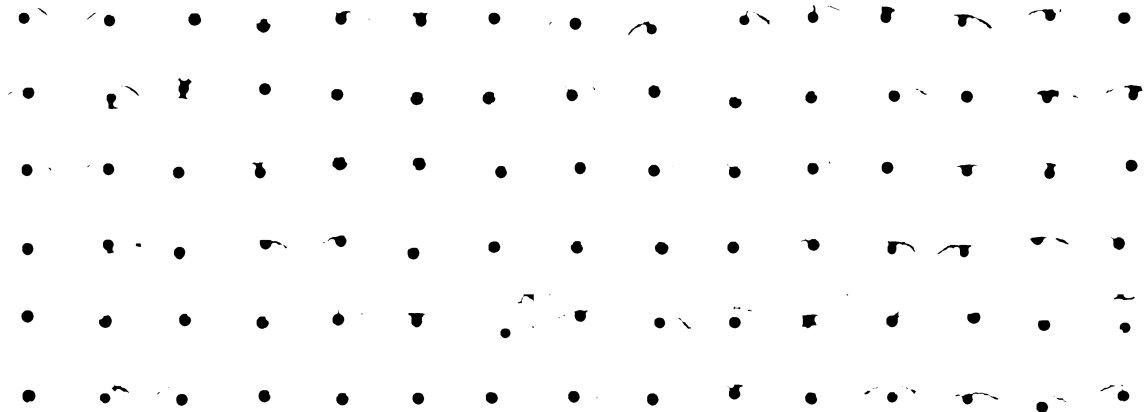
3.2.1 Zamknięcie

Na tym etapie program ma obrazy binarne, na których pozostały jedynie źrenice. W celu wypełnienia dziur w otrzymanych źrenicach oraz usunięcia małych artefaktów bez zmiany rozmiaru źrenic i ich kształtu wykorzystałem operacje morfologiczne.

Chcąc zachować rozmiar i kształt źrenic miałem do wyboru jedynie operację zamknięcia i otwarcia. Jednocześnie mając na uwadze fakt, że te operacje są idempotentne wywnioskowałem, że na obrazie binarnym przeprowadzę jednokrotnie operację zamknięcia i otwarcia. Pozostała do określenia kolejność tych zabiegów. Operacja otwarcia na źrenicy z dziurami może powodować powiększenie się dziur, a celem jest uzyskanie samej źrenicy. Zatem pierwszym krokiem powinno być zastosowanie zamknięcia na tyle dużego, aby wyeliminować dziury w źrenicy, ale jednocześnie jak najmniejszego, aby nie uwypuklić ewentualnych artefaktów wokół źrenicy.

Aby algorytm działał na różnorodnych obrazach wielkość elementu strukturalnego określiłem jako procent szerokości obrazu w pikselach. Co do kształtu jądra wybrałem koło, co wydaje się być naturalnym wyborem z uwagi na to, że nie chcemy kłaść większej wagi na żaden z kierunków.

Eksperymentalnie wyznaczyłem optymalną wielkość jądra na 10% szerokości obrazu. Punkt odniesienia był po środku elementu strukturalnego.

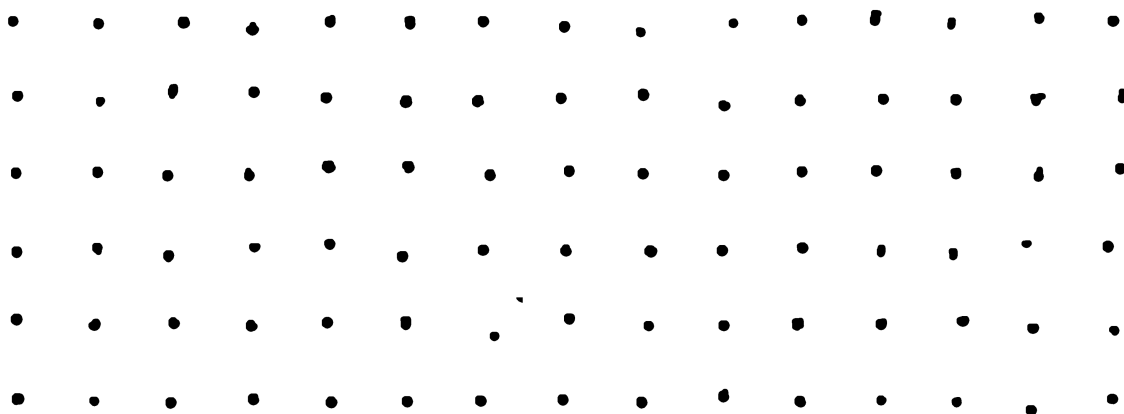


Rysunek 7: Obrazy binarne po operacji zamknięcia z elementem strukturalnym w kształcie koła, rozmiarze 10% szerokości zdjęcia w pikselach i z punktem odniesienia w środku

3.2.2 Otwarcie

Po operacji zamknięcia dziury w żrenicy zostały wypełnione. Następnym krokiem jest zastosowanie otwarcia w celu usunięcia artefaktów. Tutaj im większy rozmiar elementu strukturalnego, tym więcej artefaktów może usunąć, ale zbyt duże jądro może również wyeliminować samą żrenicę.

Eksperymentalnie wyznaczyłem optymalną wielkość jądra na 10% szerokości obrazu. Punkt odniesienia był po środku elementu strukturalnego w kształcie koła.

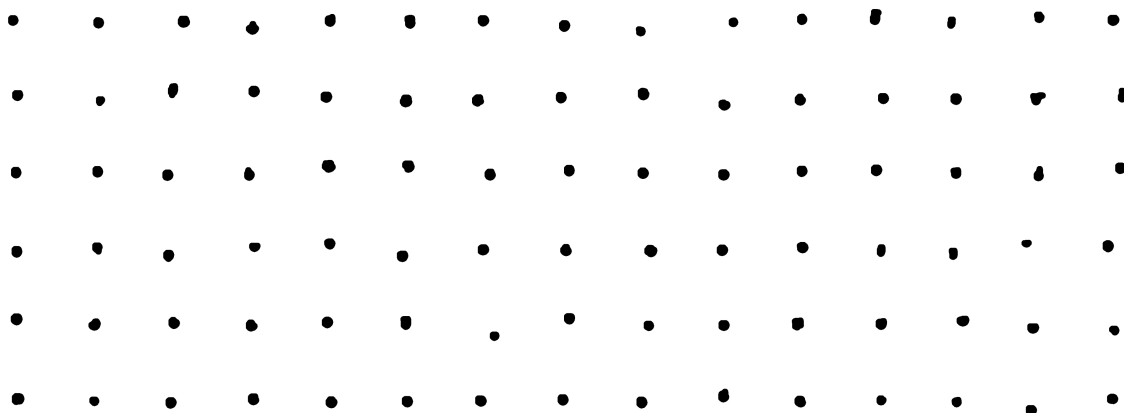


Rysunek 8: Obrazy binarne po operacji otwarcia z elementem strukturalnym w kształcie koła, rozmiarze 10% szerokości zdjęcia w pikselach i z punktem odniesienia w środku

3.2.3 Oczyszczenie obrazu

Czasem na obrazach były obecne brwi lub inne ciemniejsze elementy na brzegach zdjęcia, które pozostawały po zastosowaniu wcześniejszych operacji morfologicznych. Takie pozostałości mogą mieć wpływ na projekcje w następnym etapie. W celu wyeliminowania tych artefaktów zmieniłem piksele na białe, jeśli były w pewnej odległości od środka zdjęcia.

Eksperymentalnie wyznaczyłem optymalną część pozostawienia pikseli w zależności od odległości od środka obrazu na 75% szerokości dla osi x i 75% wysokości dla osi y.



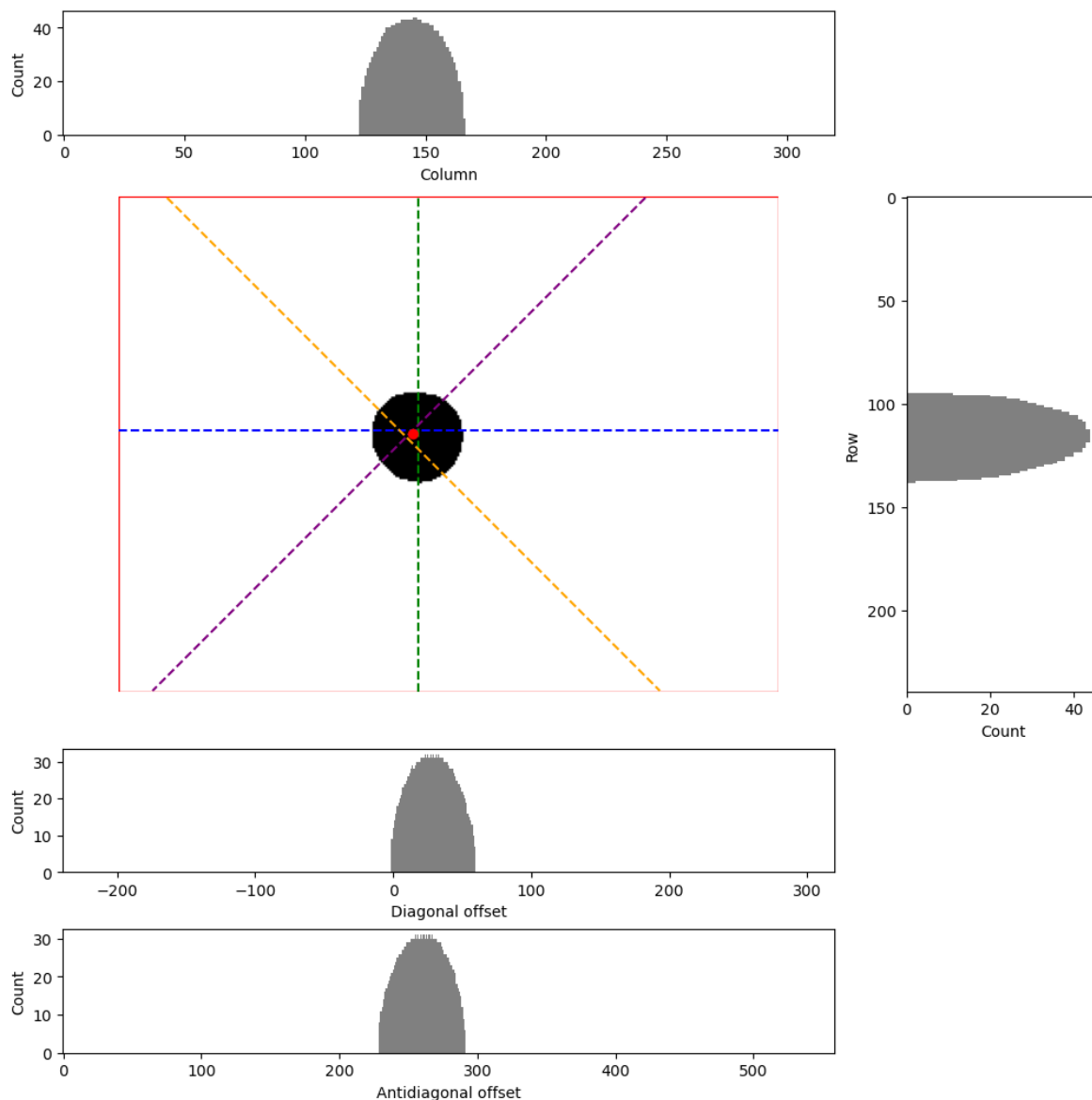
Rysunek 9: Obrazy binarne po zmianie pikseli na białe w zależności od odległości od środka obrazu: $>75\%$ szerokości dla osi x i $>75\%$ wysokości dla osi y

3.3 Projekcje

Mając obraz binarny z samą źrenicą posłużyłem się projekcjami w celu znalezienia środka i promienia źrenicy.

Środek wyznaczałem jako średni punkt przecięcia się linii poprowadzonych z indeksów (odpowiadającym pikselom w danej linii) z maksymalną wartością dla każdej z projekcji; poziomej, pionowej, diagonalnej i antydiagonalnej.

Promień źrenicy wyznaczyłem jako średnia z szerokości obszaru występowania czarnych pikseli na histogramie projekcji dla wszystkich poprzednio wspomnianych projekcji.



Rysunek 10: Projekcja wybranego obrazu binarnego źrenicy. Górny histogram przedstawia projekcję pionową, niżej po prawej stronie jest projekcja pozioma, a na samym dole od góry jest projekcja diagonalna i antydiagonalna. Po środku jest obraz binarny źrenicy z narysowanymi liniami od indeksów pikseli z maksymalnymi wartościami projekcji. Czerwony punkt oznacza środek źrenicy i jest wyznaczony jako średni punkt przecięcia wszystkich linii.

3.4 Przeniesienie na inny obraz i wyniki

Wyznaczony został już środek źrenicy i jej promień. Do tej pory algorytm operował na potencjalnie pomniejszonym obrazie. Aby móc w przyszłości pozyskać kod tęczówki z obrazu, gdzie tęczówka jest w lepszej jakości, konieczne jest wyznaczenie granicy tęczówki również na początkowym rozmiarze. Zadanie jest proste, bo można skorzystać z faktu, że oba obrazy mają te same proporcje. Tutaj musiałem jedynie odpowiednio przeskalować znalezione wcześniej granie źrenic.



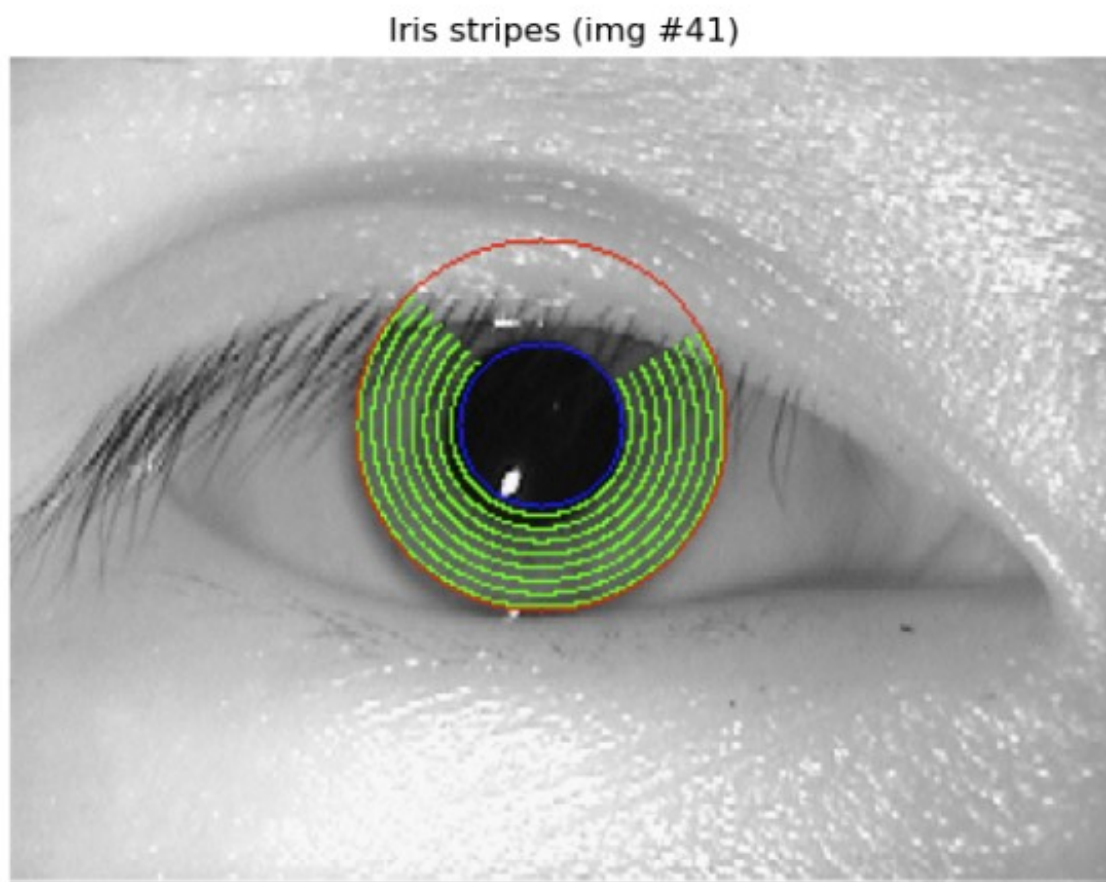
Rysunek 11: Granice źrenic naniesione na oryginalne obrazy

4 Wykrywanie tęczówki

4.1 Nieudana próba

Wykrycie granicy tęczówki nie było trywialne. Problem stanowił fakt, że na zdjęciach, poza obecnymi rzęsami i odbłaskami, często część tęczówki jest zakryta źrenicą. Dodatkowo tęczówki mają różne kolory, a po konwersji do odcieni szarości - różne jasności.

Moim pierwszym pomysłem było stworzenie maski, która by nie brała pod uwagę powiek. Niestety ten pomysł nie miał przyszłości, ponieważ celem jest pozyskanie kodu tęczówki, a branie różnych fragmentów tęczówek dla różnych zdjęć znacząco utrudni ich późniejsze porównywanie na podstawie kodu.



Rysunek 12: Wyznaczenie pasów do segmentacji tęczówki przy ominięciu fragmentu z górną powieką

Dlatego ostatecznie użyłem na każdej z tęczówek tej samej segmentacji na 8 radialnych pasów. Dokładniej metoda jest omówiona w kolejnych sekcjach.

4.2 Binaryzacja

4.2.1 Oczyszczenie obrazu

Podobnie jak dla źrenicy, w pewnej odległości od środka obrazu piksele są zmieniane na białe. Tutaj w osi x pozostawiam obraz bez zmian, natomiast przycinam go do 75% w osi y. Celem jest wyeliminowanie ciemniejszych fragmentów np. brwi, które nie stanowiły problemu dla źrenicy, ale w przypadku tęczówki są często ciemniejsze i pozostają po binaryzacji.

4.2.2 Adaptacyjny próg binaryzacji

Tak samo jak w przypadku binaryzacji dla źrenic, dla tęczówek korzystam z adaptacyjnej metody doboru parametru określającego próg binaryzacji na podstawie frakcji pikseli białych to całkowitej liczby pikseli. Pozostawienie elementów takich jak brwi pogorszyłoby działanie tej metody, ponieważ brew może zajmować znaczącą część obrazu. Dlatego oczyszczenie zdjęć w kierunku osi y przyniosło pozytywne efekty.

Celem na tym etapie nie było uzyskanie jak najpełniejszych tęczówek.

Frakcję pikseli białych wyznaczyłem eksperymentalnie na 0.75.



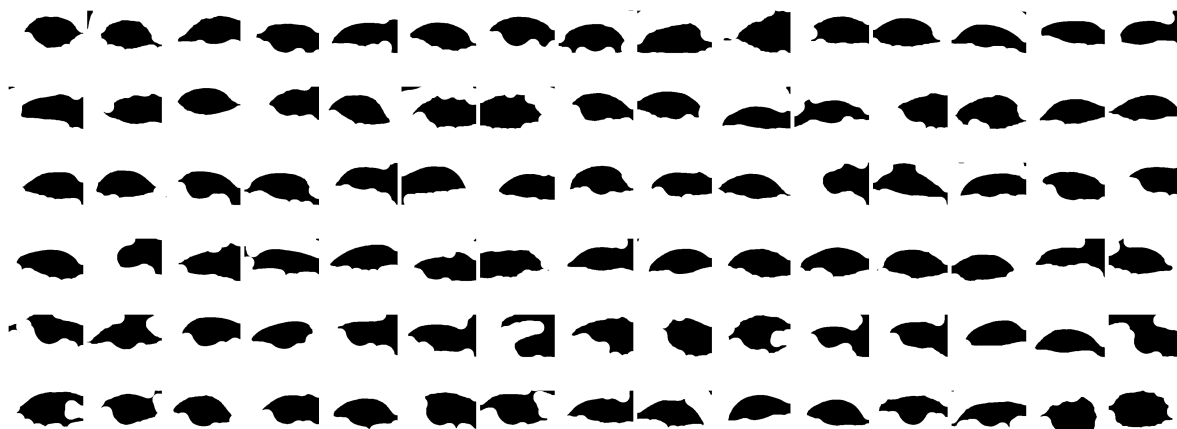
Rysunek 13: Binaryzacja adaptacyjną wartością progu dla maksymalnej frakcji białych pikseli równej 0.75

4.3 Operacje morfologiczne

4.3.1 Zamknięcie

W przypadku tęczówki podejście miałem inne, niż dla źrenicy. Zamiast posługiwać się projekcjami to wyznaczania granicy tęczówki chciałem zrobić to badając tempo wzrostu średniej jasności w kierunku radialnym od środka źrenicy (założyłem, że środek tęczówki i źrenicy są w tym samym punkcie). Takie podejście ma jednak problem; skóra (powieki) są jaśniejsze od tęczówki, i po napotkaniu na ten obszar jasność wzrasta. To może prowadzić do mylnych wniosków, bo promień tęczówki zależy wtedy od poziomu otwarcia oka na zdjęciu. Aby temu zapobiec postanowiłem liczyć jasność jedynie na powierzchni oka (źrenica, tęczówka i ewentualnie twardówka). Tutaj chciałem posłużyć się dużym elementem strukturalnym w celu stworzenia obrazu binarnego w kształcie oka; źrenica z tęczówką i czasem z twardówką, ale bez powiek. Taki obraz później wykorzystałem jako maskę definiującą obszar, na którym należy liczyć wzrost jasności.

Eksperymentalnie wyznaczyłem optymalną wielkość jądra na 20% szerokości obrazu. Punkt odniesienia był po środku elementu strukturalnego w kształcie koła.

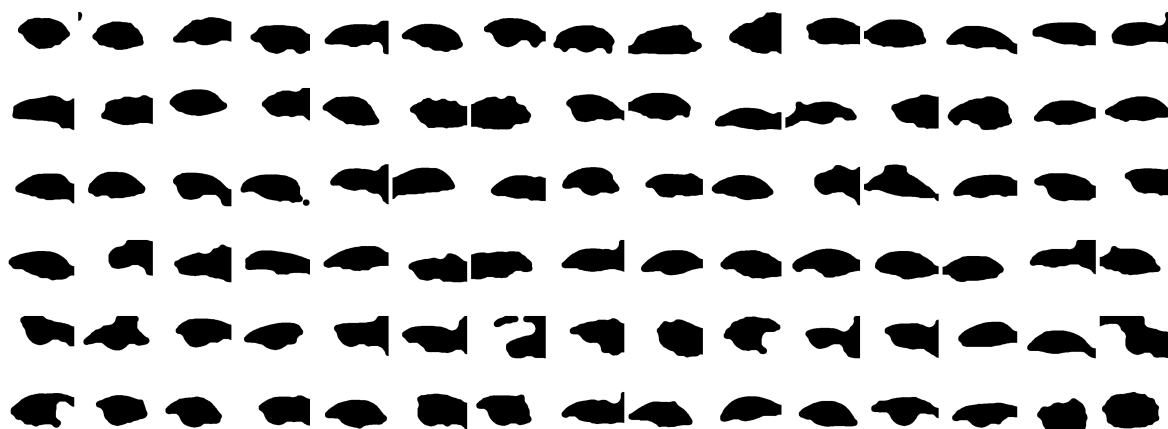


Rysunek 14: Obrazy binarne po operacji zamknięcia z elementem strukturalnym w kształcie koła, rozmiarze 20% szerokości zdjęcia w pikselach i z punktem odniesienia w środku

4.3.2 Otwarcie

Ta operacja wygładzała kształt oka powstały w poprzednim kroku. Granica oka nie zawsze była dokładnie zdefiniowana, np. przez rzęsy, które pozostawały po binaryzacji. Otwarcie miało na celu wyeliminowanie tych niedoskonałości, przybliżając tym samym kształt oka.

Eksperymentalnie wyznaczyłem optymalną wielkość jądra na 10% szerokości obrazu. Punkt odniesienia był po środku elementu strukturalnego w kształcie koła.



Rysunek 15: Obrazy binarne po operacji otwarcia z elementem strukturalnym w kształcie koła, rozmiarze 10% szerokości zdjęcia w pikselach i z punktem odniesienia w środku

4.4 Badanie pochodnej jasności

Mając wyznaczony środek źrenicy buduję okręgi o tym samym środku i różnym promieniu. Różnica długości promieni kolejnych okręgów jest stała. Te okręgi wyznaczają obszary, na których obliczana jest średnia jasność pikseli. Zmiana tak obliczonej średniej jasności przy przejściu na kolejny okrąg jest interpretowana jako pochodna. Szukam miejsca, gdzie pochodna ma największą wartość (wzrost średniej jasności jest największy) i oznaczam ten okrąg jako

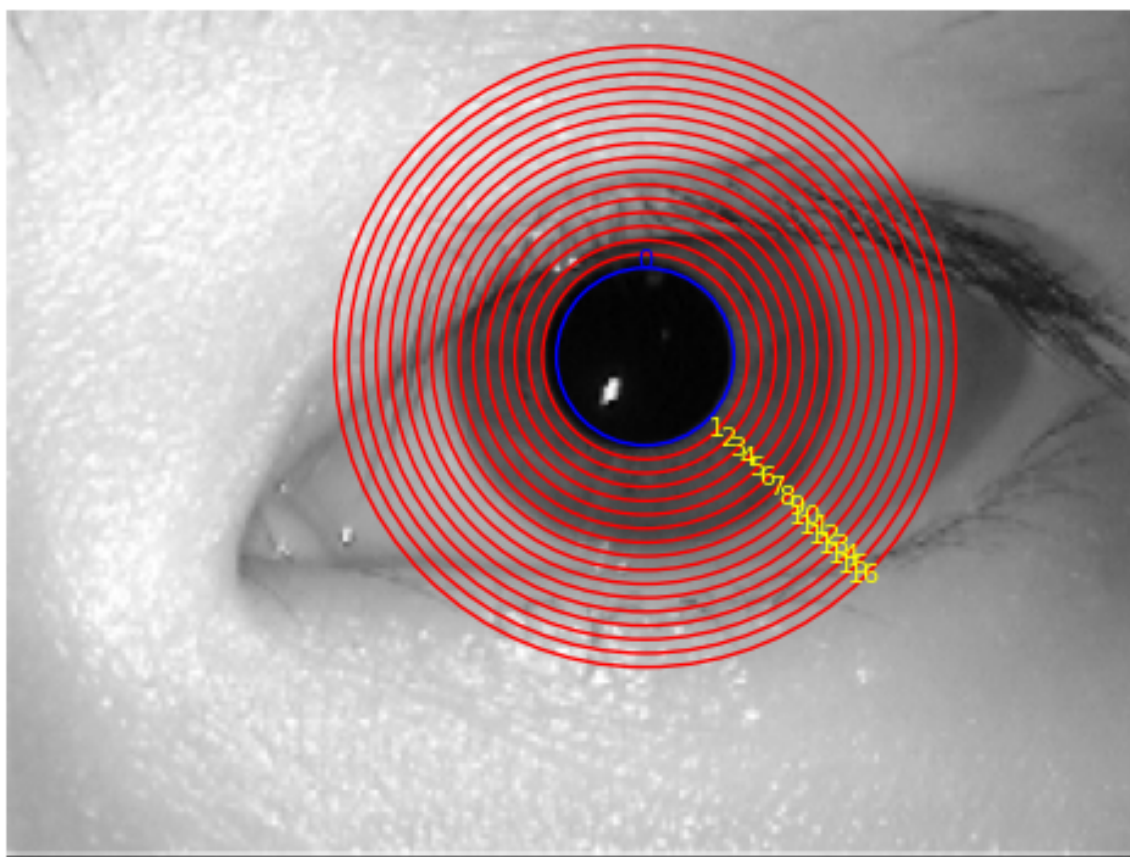
zewnątrzną granicę tęczówki.

Jak wspomniałem wcześniej, powieka stanowi obszar jaśniejszy od elementów oka i może zaburzać notowania wartości pochodnych tym samym mylnie stawiając hipotezę o umiejscowieniu granicy tęczówki. Aby temu zapobiec okręgi liczą tempo wzrostu średniej jasności jedynie na masce zdefiniowanej poprzez binaryzację i operacje morfologiczne na obrazie.

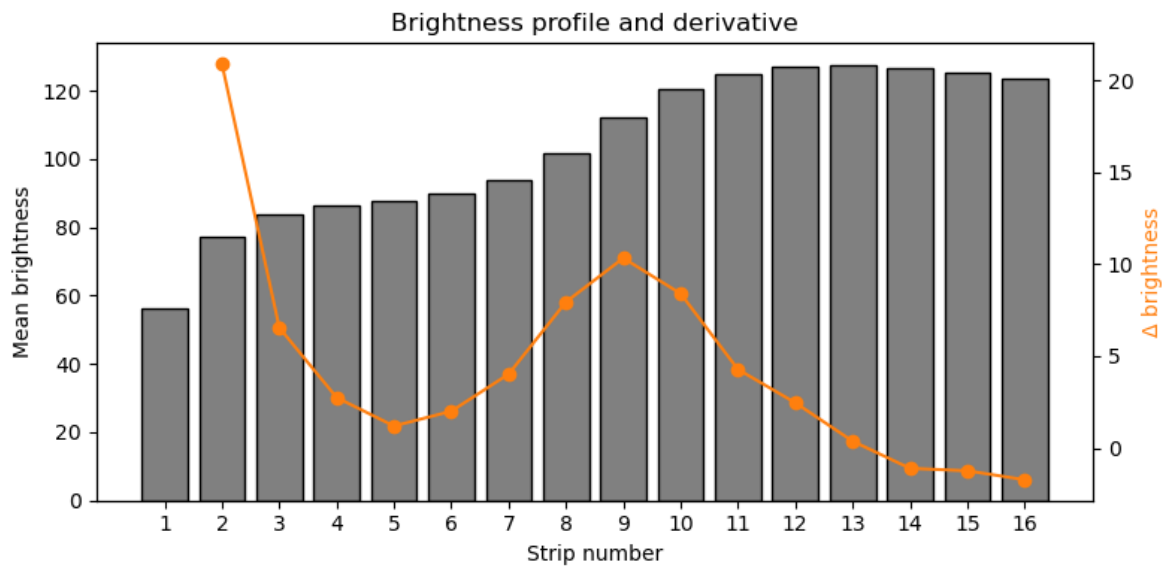
Pierwszy okrąg zaczyna się w miejscu wyznaczonej granicy źrenicy. Czasem z tego powodu często największa wartość pochodnej jest na pierwszym lub drugim okręgu. Dlatego pod uwagę są brane są okręgi począwszy od drugiego. Ten parametr jak i liczbę okręgów można modyfikować.

Eksperymentalnie wyznaczyłem, że dobre wyniki są przy liczbie okręgów równej 16 i zaczynając od drugiego okręgu.

Image #18: pupil+16 rings



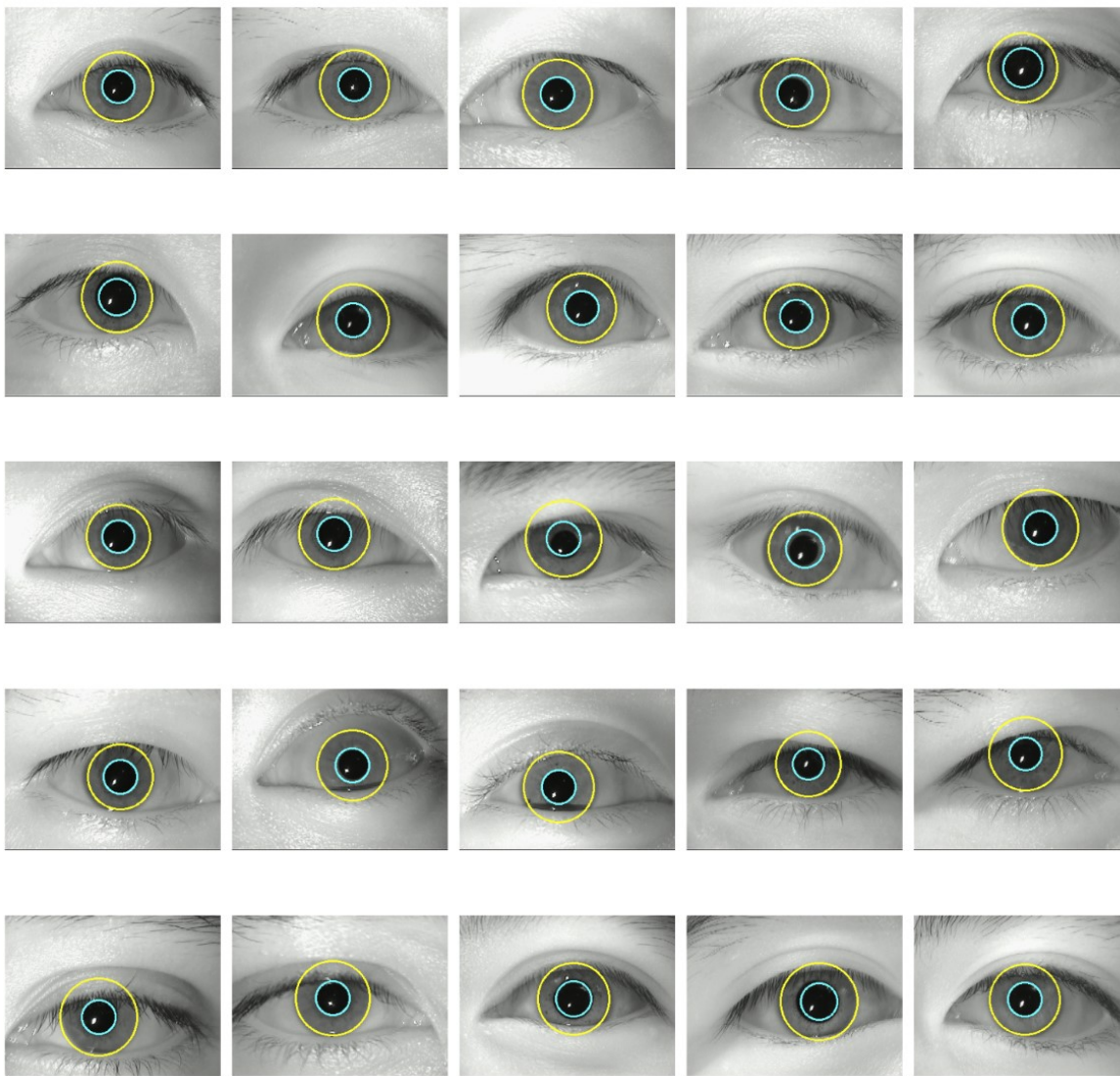
Rysunek 16: Nałożenie na obraz oka w odcieniu szarości 16 okręgów. Okrąg granatowy to wyznaczona wcześniej granica źrenicy.



Rysunek 17: Wysokość słupków to wartości średniej jasności dla danego okręgu (oś x). Nałożone punkty o kolorze pomarańczowym to wartości pochodnej (wartość wzrostu średniej jasności).

4.5 Przeniesienie na inny obraz i wyniki

Wyznaczony został już środek źrenicy, jej promień oraz promień tęczówki. Odpowiednio skalując uzyskane wyniki granice źrenicy i tęczówki można nanieść na obraz przed ewentualnym zmniejszeniem jego rozmiaru.



Rysunek 18: Granice źrenic i tęczówek naniesione na oryginalne obrazy

5 Kod tęczówki

5.1 Metodologia

Na tym etapie chciałem skupić się jedynie na pozyskiwaniu kodu z tęczówki algorytmem Daugmana, dlatego jako obrazu, na którym operowałem, użyłem ręcznie wyizolowanej tęczówki z dodatkowego zbioru danych. W ten sposób dotychczas napisany algorytm był w stanie z dużą dokładnością określić granice źrenicy i tęczówki. Operowanie na zdjęciach ze zbioru *MMU iris dataset* dałoby wyniki trudniejsze do walidacji, ponieważ zdjęcia są mniejsze i nie są w kolorze - wzory tęczówki są trudniejsze do rozpoznania.



Rysunek 19: Wycinek obrazu użyty podczas implementacji algorytmu Daugmana

5.2 Radialne pasy

Na każdej tęczówce jest umieszczanie 8 radialny pasów. To są wycinki pierścieni o środku w środku źrenicy, zajmujące na szerokość przestrzeń tęczówki. 4 pierwsze pasy (licząc od środka) mają na górze wycinek 30 stopni, kolejne 2 mają różne wycinki na górze i na dole, a łącznie zajmują powierzchnię 226 stopni. Ostatnie 2 pierścienie również mają równe wycinki na górze i na dole, a łącznie zajmują powierzchnię 180 stopni. Taki podział był jednakowy dla każdej tęczówki, dzięki czemu porównanie różnych kodów tęczówek stawało się łatwiejsze, a dodatkowo wycinki na górze i na dole oka umiejętnie omijały w większości przypadków fragmenty zasłonięte przez powieki.

Image #1

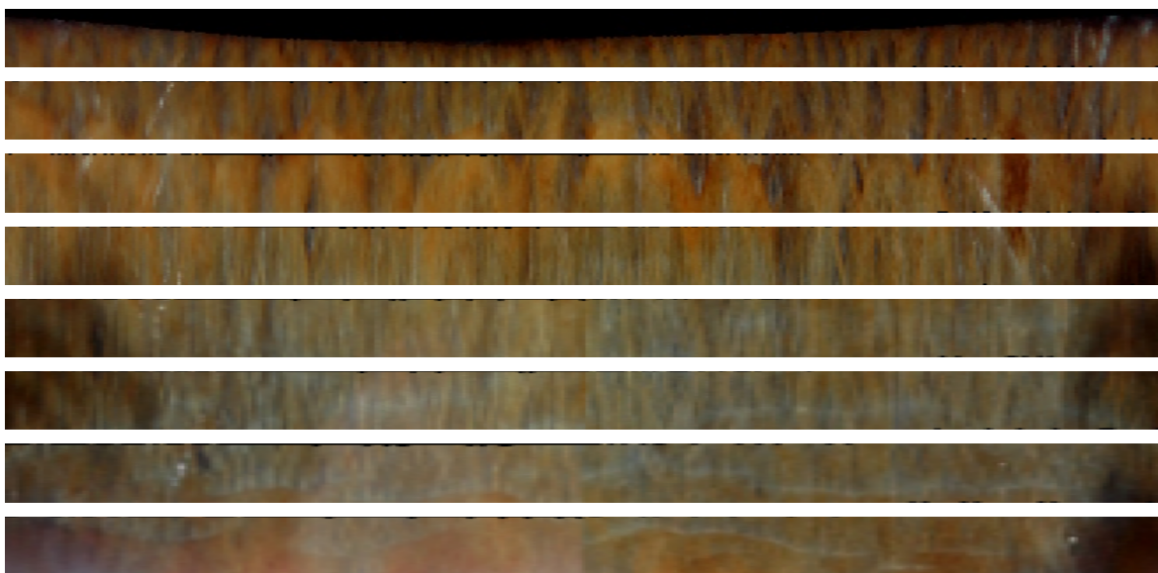


Rysunek 20: Wizualizacja 8 radialnych pasów nałożonych na tęczówkę

5.3 Rozwinięcie pasów

Wyznaczone w poprzednim kroku 8 pasów następnie rozwijałem do postaci prostokątnych, zmieniając współrzędne biegunowe, na kartezjańskie.

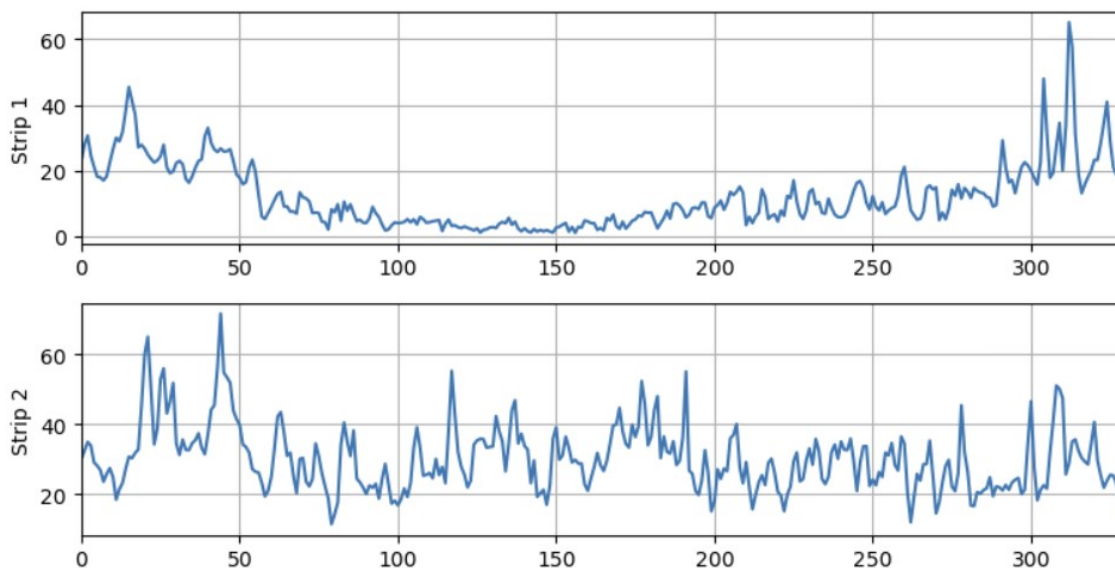
Na ogół wszystkie pasy miały podobną długość, choć różniącą się. Ich wysokość była równa. Każdemu pasowi przed rozwinięciem była przyporządkowywana współrzędna w zależności od położenia: najbliższej źrenicy pas dostawał wartość 0, najdalej źrenicy - 1, a pasy pomiędzy dostawały odpowiednie wartości pośrednie. Te współrzędne po rozwinięciu określały wysokość położenia pasu przy przedstawieniu rozwiniętej tęczówki jako prostokąta. Poniżej zamieszczona jest właśnie taka wizualizacja, gdzie dodany został odstęp między kolejnymi pasami, aby ułatwić ich rozróżnienie.



Rysunek 21: 8 pasów tęczówki rozwiniętych do prostokątów. Pasy były różnej długości (choć podobnej), więc na potrzeby wizualizacji niektóre zostały rozciągnięte.

5.4 Kompresja pasów do jednego wymiaru

Aby zredukować pasy do jednowymiarowych tablic wartości uzyskane 8 pasów było konwertowanych do odcieni szarości, a następnie za pomocą okna Gaussa w każdej kolumnie (okno Gaussa preferuje wartości po środku kolumn) wartości były uśredniane do jednej liczby.



Rysunek 22: Jednowymiarowa reprezentacja dwóch pierwszych pasów po kompresję oknem Gaussa

5.5 Transformacja falkami Gabora

Po uzyskaniu jednowymiarowych sygnałów $s_k(n)$ (gdzie $k = 1, \dots, 8$ indeksuje pasy, a n – pozycję wzdłuż prostej) następnym krokiem było wydobywanie z nich cech częstotliwościowo-fazowych przy użyciu falki Gabora.

Kluczową cechą metody Daugmana jest wykorzystanie informacji fazowej, ponieważ jest bardziej odporna na zmiany oświetlenia i kontrastu.

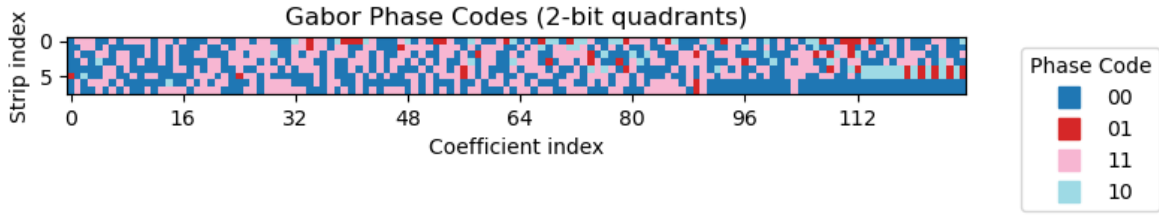
W implementacji przyjąłem częstotliwość jako parametr do wyboru f przy stałym $\sigma = \frac{1}{2}\pi f$, dzięki czemu zadanie dekompozycji miało dwa stopnie swobody; położenie i częstotliwość.

5.6 Otrzymanie kodu tęczy

Otrzymane zespolone współczynniki były następnie konwertowane do ciągu bitowego według przynależności do ćwiartek płaszczyzny zespolonej:

$$\begin{cases} \text{Ćwiartka I: } \Re(c) \geq 0, \Im(c) \geq 0 & \mapsto (0,0), \\ \text{Ćwiartka II: } \Re(c) < 0, \Im(c) \geq 0 & \mapsto (0,1), \\ \text{Ćwiartka III: } \Re(c) < 0, \Im(c) < 0 & \mapsto (1,1), \\ \text{Ćwiartka IV: } \Re(c) \geq 0, \Im(c) < 0 & \mapsto (1,0). \end{cases}$$

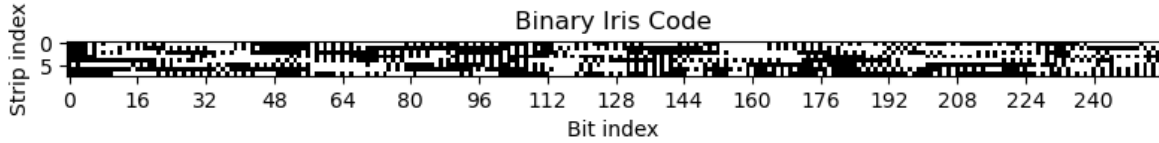
Poniżej przedstawiłem binarny kod tęczy dla różnych wartości częstotliwości f .



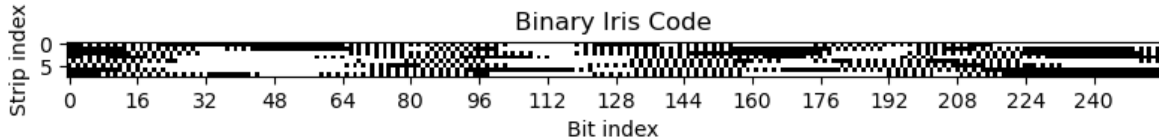
Rysunek 23: Kod tęczy na przestrzeni liczb zespolonych dla wartości $f = \frac{1}{2}$



Rysunek 24: Binarny kod tęczy dla wartości $f = \frac{1}{2}$



Rysunek 25: Binarny kod tęczy dla wartości $f = 1.25$

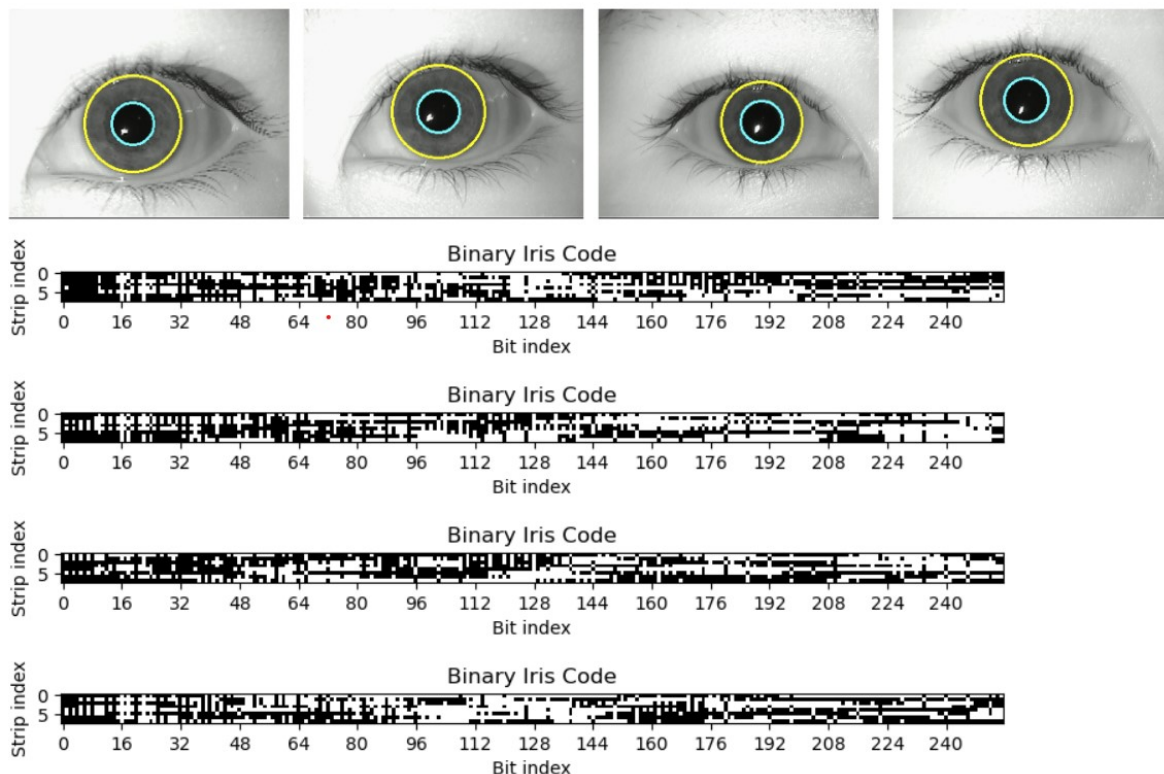


Rysunek 26: Binarny kod tęczy dla wartości $f = 0.47 \times \pi$

6 Wynik

6.1 Przedstawienie wyniku

Poniżej przedstawiłem wyniki dla czterech obrazów ze zbioru *MMU iris dataset* dla częstotliwości $f = 0.47 \times \pi$.



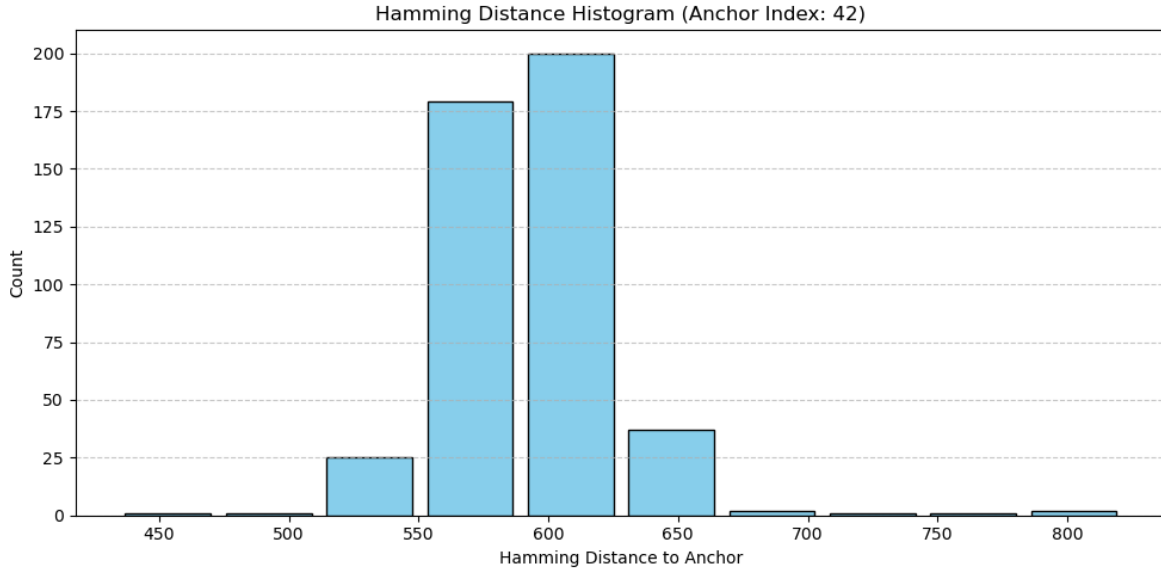
Rysunek 27: Wynik działania algorytmu dla domyślnych parametrów i częstotliwości $f = 0.47 \times \pi$

6.2 Badanie odległości Hamminga

W cel porównania kodów tęczówek posłużyłem się odległością Hamminga.

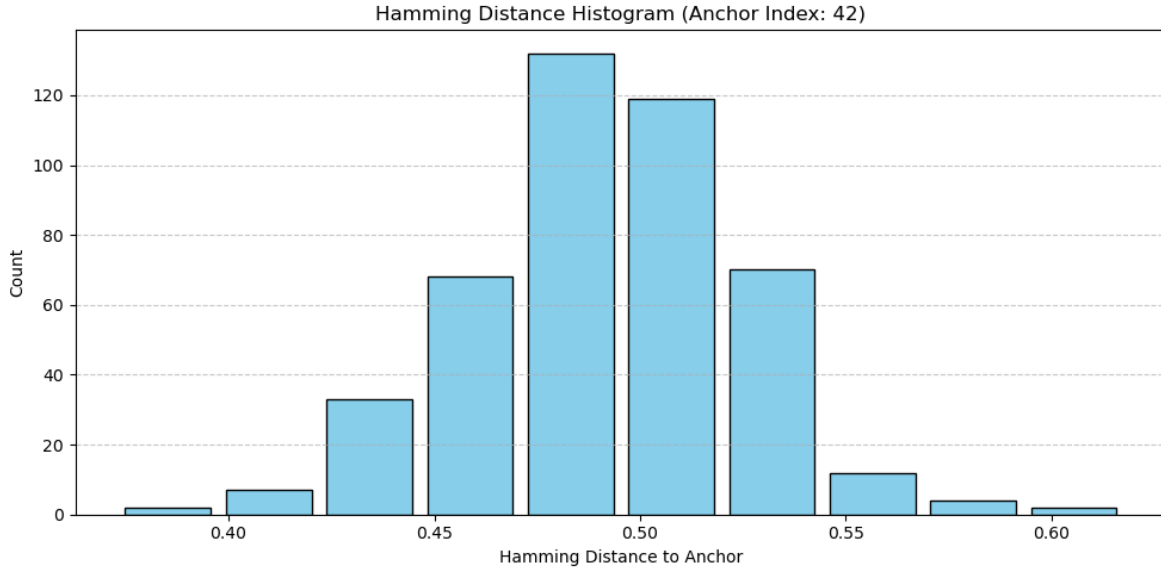
W zbiorze *MMU iris dataset* każda tęczówka była obecna na 5 obrazach. Porównując średnią wartość odległości Hamminga dla tych samych tęczówek na różnych zdjęciach i dla różnych tęczówek można wywnioskować, że faktycznie kod tęczówki uzyskany zaimplementowanym algorytmem niesie ze sobą informacje pozwalające za rozpoznanie tęczówki, choć różnica między kodami tej samej tęczówki, a innymi, jest niewielka.

Następujące wyniki były dla parametry $f = \frac{1}{2}$. Średnia odległość Hamminga dla tych samych tęczówek wynosiła 580 z odchyleniem 63, a dla innych tęczówek 595 z odchyleniem 47.



Rysunek 28: Histogram odległości Hamminga między jednym wybranym obrazem, a wszystkimi dla parametru $f = \frac{1}{2}$

Jakość uzyskanego kodu w dużej mierze zależy, poza jakością zdjęć, od parametru f . Poniżej przedstawiłem histogram dla innej wartości częstotliwości oraz przy znormalizowaniu odległości Hamminga (po podziale przez 2048 - maksymalna odległość Hamminga dla naszego kodu tęczy, bo mamy 8 pasów, każdy jest zakodowany ostatecznie 128×2 bitami).



Rysunek 29: Histogram znormalizowanej odległości Hamminga między jednym wybranym obrazem, a wszystkimi dla parametru $f = 0.47 \times \pi$

7 Aplikacja

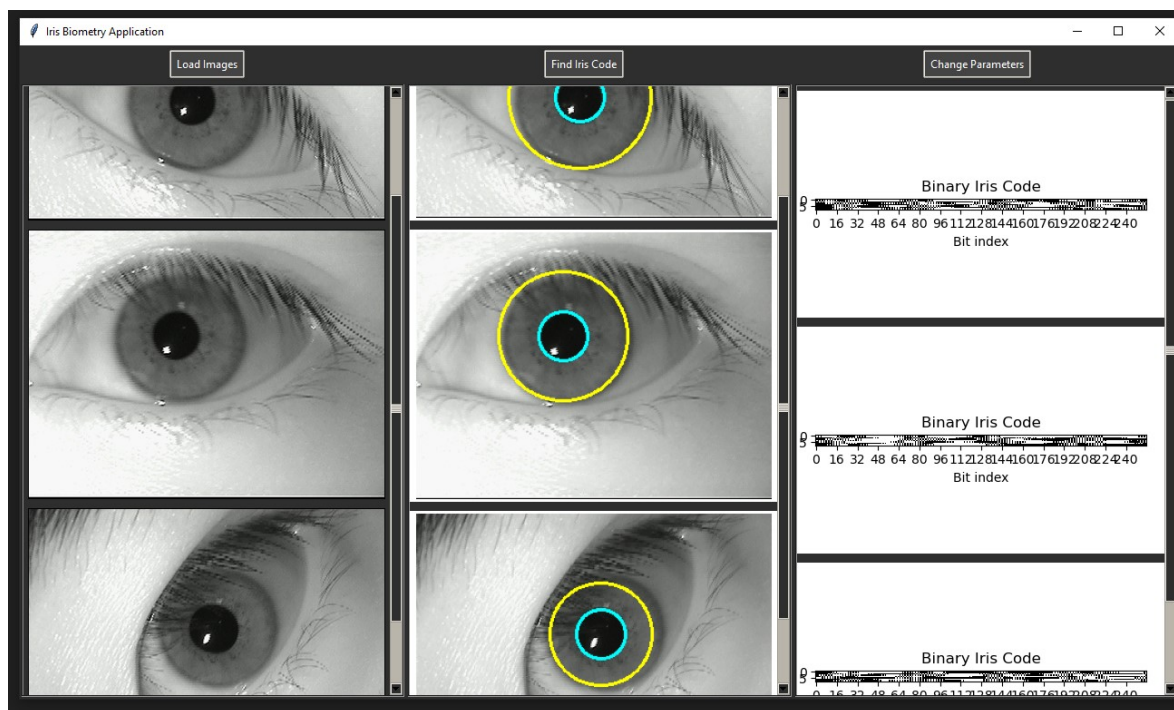
Opisywany do tej pory algorytm umieściłem w aplikacji w formie przyjaznej użytkownikowi. Program napisałem w Pythonie korzystając z modułu *tkinter*.

W aplikacji można wyróżnić 2 główne obszary; pasek górny z przyciskami i panel pod paskiem.

Na pasku górnym znajdują się kolejno; przycisk do wczytywania zdjęć do aplikacji, przycisk do pozyskania kodów ze wczytanych zdjęć oraz przycisk do zmiany parametrów w algorytmie.

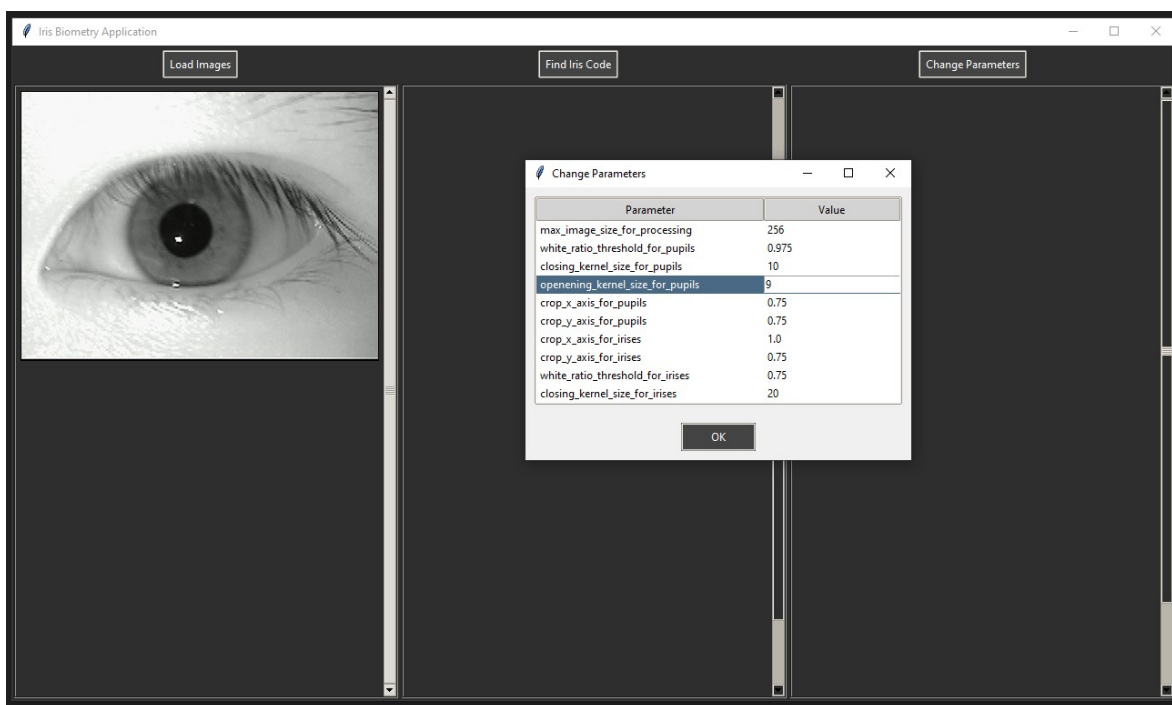
Dolny panel składa się z trzech mniejszych paneli, które służą kolejno do wyświetlania wczytanych obrazów, pokazania znalezionej granicy źrenicy i tęczówki nałożonych na oryginalne obrazy oraz do wyświetlenia znalezionego kodów tęczówki. Dwa ostatnie panele są puste dopóki użytkownik nie kliknie przycisku uruchamiającego algorytm.

Aplikacja pozwala na wczytanie i przetwarzanie wielu zdjęć na raz.



Rysunek 30: Przedstawienie wyglądu interfejsu aplikacji i wyników działania na przykładowych obrazach

Użytkownik może swobodnie modyfikować parametry metody, w tym częstotliwość falek Gabora, korzystając z dodatkowego okna, które się pojawia po kliknięciu w przycisk do zmiany parametrów.



Rysunek 31: Wygląd aplikacji przy zmianie parametrów algorytmu

8 Podsumowanie

Celem projektu było opracowanie metody wyznaczania granic tęczówek i zaimplementowanie algorytmu Daugmana do pozyskiwania ze znalezionych tęczówek kodów binarnych. Program miał działać poprawnie za zbiorze zdjęć oczu *MMU iris dataset*.

Projekt realizowałem w języku Python i podzieliłem proces twórczy na etapy, w których skupiałem się na jednym zagadnieniu, którego zrealizowanie przybliżało mnie do ostatecznego celu.

Aby ułatwić operowanie na zdjęciach najpierw obrazy były zmniejszane mając na celu szybsze działanie algorytmu, a następnie przekształcane na odcienie szarości. Dodatkowo użyłem na obrazach kilka mniejszych przetworzeń w celu wygładzenia i usunięcia odbłasków. Podczas całego procesu twórczego miałem na uwadze adaptacyjność algorytmu do różnych zdjęć, więc ustalając parametry w kodzie, uzależniałem je od cech danych wejściowych.

Pierwszym etapem była lokalizacja i wyznaczenie granic źrenicy. Za pomocą binaryzacji z progiem dobieranym w sposób adaptacyjny zależnie od końcowej proporcji białych pikseli otrzymałem obrazy z czarnym kołem w miejscu źrenicy. Następnie w celu oczyszczenia obrazu z artefaktów zastosowałem operacje morfologiczne; najpierw zamknięcie, aby wypełnić dziury w źrenicy, potem otwarcie, aby pozbyć się mniejszych pozostałości. Dodatkowo usuwałem czarne elementy z daleka od środka obrazu, które na pewno nie były częścią oka z uwagi na charakter używanego zbioru danych. Na końcu użyłem projekcji poziomej, pionowej, diagonalnej i antydiagonalnej do znalezienia środka i promienia źrenicy.

Kolejnym krokiem było wyznaczenie granicy tęczówki. To zadanie było trudniejsze od lokalizacji źrenicy z uwagi na różną jasność tego elementu oka; zwykła binaryzacja nie jest w stanie uchwycić tęczówki w wielu przypadkach bez jednoczesnego przepuszczania niechcianych elementów zdjęcia. Tutaj założyłem, że środek źrenicy i tęczówki jest tym samym punktem szukałem granicy tęczówki badając tempo zmiany średniej jasności pikseli na obszarze wyznaczonym przez radialne pierścienie o jednostajnie rosnących promieniach i środku w centrum źrenicy, zaczynając od granicy źrenicy. Największy wzrost mierzonej wartości miał być na granicy tęczówki i twardówki. Przeszkodą w tej metodzie stanowił fakt, że powieki często nachodziły na tęczówkę, a z uwagi na ich jasność wyniki badania tempa wzrostu średniej jasności wspomnianymi pierścieniami mogły by mylnie określać granicę tęczówki. Aby temu zaradzić, badałem jasność pikseli jedynie na wcześniej utworzonej masce. Maskę była konstruowana binaryzacją i operacjami morfologicznymi, jak w przypadku źrenicy, jednak z odpowiednio zmienionymi parametrami, aby otrzymać pełną tęczówkę, nawet jeśli w wyniku tego procesu przepuszczane były również inne fragmenty obrazu - kluczem było wykluczenie z maski powiek jako najjaśniejszych obszarów zaburzających wyniki.

Wyznaczone granice źrenicy i powieki były nanoszone na obrazy o oryginalnej wielkości przed przetworzeniem, aby kod tęczówki był pozyskiwany z możliwie najbardziej klarownej grafiki.

Ostatnim etapem było wyznaczenie kodu tęczówki algorytmem Daugmana. Na tęczówce było umieszczane osiem radialnych pasów z odpowiednimi wycinkami na górze i na dole, aby uniknąć włączenia powiek, rzęs i ich cieni. Następnie pasy były rozwijane do prostokątów i kompresowane do postaci jednowymiarowej za pomocą średniej liczonej oknem Gaussa. Kolejnym krokiem było zastosowanie transformacji falkami Gabora i otrzymanie zespolonych

współczynników ostatecznie konwertowanych na ciągi binarne.

Cały algorytm umieściłem w przyjaznej z perspektywy użytkownika aplikacji, w której jedną z funkcjonalności jest możliwość zmiany parametrów zaimplementowanej metody.

9 Literatura

Ślot Krzysztof: Wybrane zagadnienia biometrii